GUÍA DE LABORATORIO

INFORMACIÓN BÁSICA					
ASIGNATURA:	INTRODUCCION AL DESARROLLO DE NUEVAS PLATAFORMAS (E)				
TÍTULO DE LA PRÁCTICA:	Servicio en segundo plano para sincronizar datos en Jetpack Compose				
NÚMERO DE PRÁCTICA:	10	**AÑO LECTIVO:**	2026B	**NRO. SEMESTRE:**	VIII
TIPO DE PRÁCTICA:	**INDIVIDUAL**				
	GRUPAL	**X**	**MÁXIMO DE ESTUDIANTES**		2
FECHA INICIO:	03/11/2026	**FECHA FIN:**	07/11/2026	**DURACIÓN:**	100 minutos
RECURSOS A UTILIZAR: Android Studio (Quail o superior), Kotlin, Jetpack Compose, emulador Android o dispositivo físico (API 34 o superior)					
DOCENTE(s): Mg Roxana Evelyn Limache Calatayud					

OBJETIVOS/TEMAS Y COMPETENCIAS	
OBJETIVOS: - Comprender qué es un Service en Android y la diferencia entre un servicio iniciado (started) y uno enlazado (bound). - Implementar un Service que ejecute una tarea en segundo plano sin bloquear la interfaz de usuario. - Comunicar el resultado del Service de vuelta a la interfaz Compose mediante un BroadcastReceiver.	
TEMAS: - Service: ciclo de vida y tipos (started, bound, foreground) - startService() y stopSelf(): control del ciclo de vida de un Service - Comunicación Service → UI mediante BroadcastReceiver	
COMPETENCIAS	C.a Aplica de forma flexible técnicas, métodos, principios, normas, estándares y herramientas de ingeniería necesarias para la construcción de software e implementación de sistemas de información.
	C.b Investiga nuevos modelos, metodologías, técnicas, herramientas y tecnologías por ser necesarias para mantener la vigencia en el desempeño profesional.
	C.c
	C.d

CONTENIDO DE LA GUÍA

I. MARCO CONCEPTUAL

Revisar el siguiente recurso antes de la sesión:

- Android Developers, servicios en segundo plano: https://developer.android.com/develop/background-work/services

¿Qué es un Service?

Un Service es un componente de Android diseñado para ejecutar operaciones de larga duración sin una interfaz de usuario propia, incluso si la aplicación no está visible en pantalla. A diferencia de una Activity o de un Composable, un Service no dibuja nada: solo ejecuta trabajo. Por defecto, un Service corre en el hilo principal de la aplicación, por lo que cualquier tarea pesada dentro de él debe moverse explícitamente a una coroutine o a un hilo propio para no bloquear la interfaz.

Started Service vs. Bound Service

Existen dos formas principales de trabajar con un Service. Un started Service se inicia con startService(Intent) y sigue ejecutándose de forma independiente hasta que él mismo llama a stopSelf() o alguien llama a stopService(); es el más adecuado para una tarea puntual como sincronizar datos. Un bound Service se conecta con bindService(Intent, connection), permite llamar métodos del Service directamente desde la UI, y se destruye automáticamente cuando el último cliente se desconecta con unbindService(). La Figura 1 compara ambos enfoques.

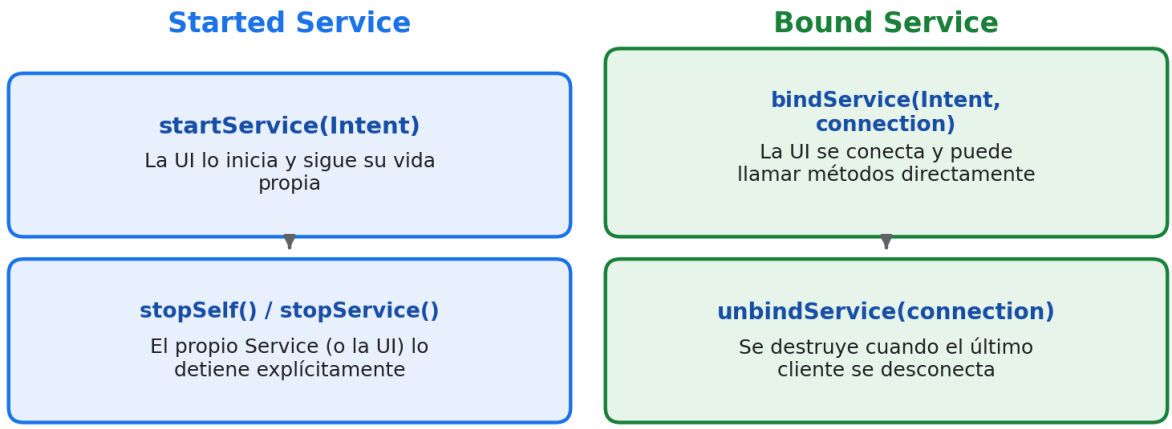


Figura 1. Comparación entre un Started Service y un Bound Service (elaboración propia).

Comunicar el resultado del Service a la UI

Un Service no puede actualizar directamente el estado de un Composable, porque vive fuera del árbol de composición. El patrón más simple para avisar a la UI que una tarea terminó es que el Service envíe un broadcast (sendBroadcast) con un Intent que incluya una acción propia y, opcionalmente, datos extra con el resultado; la UI, usando el mismo patrón de BroadcastReceiver registrado con DisposableEffect visto en el Laboratorio 04, recibe ese broadcast y actualiza su estado. La Figura 2 resume el flujo completo, desde que la UI inicia el Service hasta que el resultado aparece en pantalla.

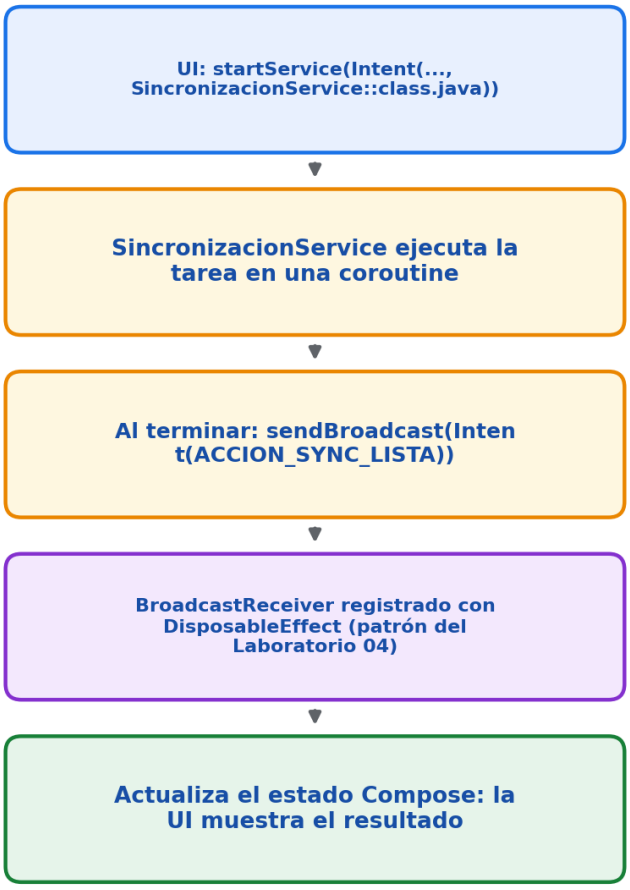


Figura 2. Flujo de comunicación entre un Service en segundo plano y la UI mediante BroadcastReceiver (elaboración propia).

II. EJERCICIO/PROBLEMA RESUELTO POR EL DOCENTE

A continuación se detallan, paso a paso, las instrucciones para implementar un Service que simula la sincronización de la lista de edificios y avisa a la UI mediante un BroadcastReceiver:

Paso 1. Crear un nuevo proyecto.

nombre sugerido para el proyecto: "SincronizacionServicio_Compose", Package name: com.example.sincronizacionservicio_compose, Language: Kotlin, Minimum SDK: el que venga por defecto en la plantilla (recuerden usar un emulador o dispositivo con API 34 o superior, según la sección de Recursos a utilizar).

Paso 2. Abrir el archivo AndroidManifest.xml y declarar el nuevo Service dentro de la etiqueta <application>:

Código (AndroidManifest.xml – fragmento dentro de <application>):

```
<service
    android:name=".SincronizacionService"
    android:exported="false" />
```

Paso 3. Agregar, debajo del package y antes de la clase MainActivity, todas las importaciones que se usarán en este ejercicio:

Código (importaciones):

```
import android.app.Service
import android.content.BroadcastReceiver
import android.content.Context
import android.content.Intent
import android.content.IntentFilter
import android.os.Bundle
import android.os.IBinder
import android.util.Log
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.Spacer
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.height
import androidx.compose.foundation.layout.padding
import androidx.compose.foundation.layout.statusBarsPadding
import androidx.compose.material3.Button
import androidx.compose.material3.CircularProgressIndicator
import androidx.compose.material3.Surface
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.runtime.DisposableEffect
import androidx.compose.runtime.getValue
import androidx.compose.runtime.mutableStateOf
import androidx.compose.runtime.remember
import androidx.compose.runtime.setValue
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.platform.LocalContext
import androidx.compose.ui.tooling.preview.Preview
import androidx.compose.ui.unit.dp
import androidx.core.content.ContextCompat
import com.example.sincronizacionservicio_compose.ui.theme.SincronizacionServicio_ComposeTheme
import kotlinx.coroutines.CoroutineScope
import kotlinx.coroutines.Dispatchers
import kotlinx.coroutines.delay
import kotlinx.coroutines.launch
```

Paso 4. Crear la clase SincronizacionService, heredando de Service, y sobrescribir onBind() devolviendo null, ya que no será un Bound Service. Por ahora dejamos pendiente la lógica de sincronización:

Código (SincronizacionService – Paso 4):

```
class SincronizacionService : Service() {

    override fun onBind(intent: Intent?): IBinder? = null

    // onStartCommand se agrega en el Paso 5
}
```

Paso 5. Sobrescribir onStartCommand(), lanzando una coroutine en Dispatchers.IO que simule una tarea de sincronización con delay(3000) y calcule un resultado simulado (por ejemplo, un número aleatorio de edificios actualizados):

Código (SincronizacionService – Paso 5):

```
class SincronizacionService : Service() {

    override fun onBind(intent: Intent?): IBinder? = null
```

```
    override fun onStartCommand(intent: Intent?, flags: Int, startId: Int): Int {
        Log.d("SYNC", "1. Servicio iniciado")
        CoroutineScope(Dispatchers.IO).launch {
            delay(3000)
            val edificiosActualizados = (1..10).random()

            // el envío del broadcast y el stopSelf() se agregan en el Paso 6
        }
        return START_NOT_STICKY
    }
}
```

Paso 6. Al terminar la coroutine, construir un Intent con una acción propia, indicar con setPackage(packageName) que el aviso es solo para esta aplicación, agregar el resultado como extra, enviarlo con sendBroadcast(), registrar un mensaje en el Logcat y llamar a stopSelf() para finalizar el Service. Con esto, SincronizacionService queda completo:

Código (SincronizacionService completo):

```
class SincronizacionService : Service() {

    override fun onBind(intent: Intent?): IBinder? = null

    override fun onStartCommand(intent: Intent?, flags: Int, startId: Int): Int {
        Log.d("SYNC", "1. Servicio iniciado")
        CoroutineScope(Dispatchers.IO).launch {
            delay(3000)
            val edificiosActualizados = (1..10).random()

            val resultado = Intent("com.example.sincronizacionservicio_compose.ACCION_SYNC_LISTA")
            resultado.setPackage(packageName)
            resultado.putExtra("edificios_actualizados", edificiosActualizados)
            sendBroadcast(resultado)
            Log.d("SYNC", "2. Aviso enviado: $edificiosActualizados edificios")

            stopSelf()
        }
        return START_NOT_STICKY
    }
}
```

Paso 7. Crear el Composable PantallaSincronizacion, obteniendo el contexto actual y una variable de estado para el resultado recibido. Por ahora dejamos el resto del cuerpo vacío; lo iremos completando en los siguientes pasos:

Código (PantallaSincronizacion – Paso 7):

```
@Composable
fun PantallaSincronizacion() {
    val context = LocalContext.current
    var resultado by remember { mutableStateOf<String?>(null) }

    // el BroadcastReceiver se agrega en el Paso 8
}
```

Paso 8. Dentro de un DisposableEffect(Unit), crear un BroadcastReceiver que en onReceive() registre un mensaje en el Logcat y actualice resultado con el extra recibido, registrarlo con ContextCompat.registerReceiver() indicando RECEIVER_NOT_EXPORTED, y desregistrarlo en onDispose { }:

Código (PantallaSincronizacion – Paso 8):

```
@Composable
fun PantallaSincronizacion() {
    val context = LocalContext.current
    var resultado by remember { mutableStateOf<String?>(null) }

    DisposableEffect(Unit) {
        val receiver = object : BroadcastReceiver() {
            override fun onReceive(ctx: Context?, intent: Intent?) {
                Log.d("SYNC", "3. Aviso recibido en la pantalla")
                val cantidad = intent?.getIntExtra("edificios_actualizados", 0) ?: 0
                resultado = "Sincronización completa: $cantidad edificios actualizados"
            }
        }
        val filtro = IntentFilter("com.example.sincronizacionservicio_compose.ACCION_SYNC_LISTA")
        ContextCompat.registerReceiver(
            context,
            receiver,
            filtro,
            ContextCompat.RECEIVER_NOT_EXPORTED
        )

        onDispose {
            context.unregisterReceiver(receiver)
        }
    }

    // el botón y el indicador de progreso se agregan en el Paso 9
}
```

Nota: *desde Android 14 (API 34), al registrar un BroadcastReceiver es obligatorio indicar si recibe avisos de otras aplicaciones (RECEIVER_EXPORTED) o solo de la propia (RECEIVER_NOT_EXPORTED). Como el aviso lo envía nuestro propio Service, usamos RECEIVER_NOT_EXPORTED; por eso el Service también indica setPackage(packageName) al enviarlo. Si se usa context.registerReceiver(receiver, filtro) sin el indicador, la aplicación se cierra al abrir la pantalla con SecurityException.*

Paso 9. Agregar un botón "Sincronizar" que inicie el Service con startService(), y un indicador de progreso que se muestre mientras resultado sea null. El Column usa statusBarsPadding() para que el contenido no quede debajo de la barra de estado. Con esto, PantallaSincronizacion queda completo:

Código (PantallaSincronizacion completo):

```
@Composable
fun PantallaSincronizacion() {
    val context = LocalContext.current
    var resultado by remember { mutableStateOf<String?>(null) }

    DisposableEffect(Unit) {
        val receiver = object : BroadcastReceiver() {
            override fun onReceive(ctx: Context?, intent: Intent?) {
                Log.d("SYNC", "3. Aviso recibido en la pantalla")
                val cantidad = intent?.getIntExtra("edificios_actualizados", 0) ?: 0
                resultado = "Sincronización completa: $cantidad edificios actualizados"
            }
        }
        val filtro = IntentFilter("com.example.sincronizacionservicio_compose.ACCION_SYNC_LISTA")
        ContextCompat.registerReceiver(
            context,
            receiver,
            filtro,
```

```
            ContextCompat.RECEIVER_NOT_EXPORTED
        )

        onDispose {
            context.unregisterReceiver(receiver)
        }
    }

    Column(
        modifier = Modifier.fillMaxSize().statusBarsPadding().padding(16.dp),
        horizontalAlignment = Alignment.CenterHorizontally
    ) {
        Button(onClick = {
            resultado = null
            context.startService(Intent(context, SincronizacionService::class.java))
        }) {
            Text("Sincronizar")
        }

        Spacer(modifier = Modifier.height(16.dp))

        if (resultado == null) {
            CircularProgressIndicator()
        } else {
            Text(resultado ?: "")
        }
    }
}
```

Nota: *desde Android 15 (API 35) las aplicaciones se dibujan de borde a borde; sin statusBarsPadding(), el botón queda debajo de la cámara y de la barra de estado del emulador y no responde bien al toque.*

Paso 10. Dentro de MainActivity, en el bloque setContent, mostrar PantallaSincronizacion:

Código (MainActivity.kt completo):

```
class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContent {
            SincronizacionServicio_ComposeTheme {
                Surface(modifier = Modifier.fillMaxSize()) {
                    PantallaSincronizacion()
                }
            }
        }
    }
}
```

Paso 11. Ejecutar la aplicación en un emulador, pulsar "Sincronizar" una vez y verificar en Logcat (filtrando con tag:SYNC) que aparece el mensaje "1. Servicio iniciado", mientras la UI muestra el indicador de progreso.

Paso 12. Esperar a que termine la coroutine (unos 3 segundos) y confirmar que en Logcat aparecen "2. Aviso enviado" y "3. Aviso recibido en la pantalla", y que la UI reemplaza el indicador por el mensaje recibido mediante el BroadcastReceiver.

III. EJERCICIOS/PROBLEMAS PROPUESTOS

Amplía el ejercicio resuelto para dar más control al usuario sobre la sincronización:

1. Agrega un botón "Detener sincronización" que llame a context.stopService(Intent(context, SincronizacionService::class.java)) mientras la tarea está en curso.

2. Modifica el Service para que, en lugar de un resultado simulado, recorra la lista de edificios persistida en Room (Laboratorio 09) y "sincronice" (por ejemplo, marque como actualizados) los que cumplan una condición, enviando el conteo real por el broadcast.
3. Registra un segundo BroadcastReceiver que escuche una acción distinta (por ejemplo "com.tuapp.ACCION_SYNC_ERROR") y simula un caso de error en el Service (por ejemplo, con una probabilidad aleatoria), mostrando un mensaje distinto en la UI.
4. Investiga qué son los WorkManager de Android y explica en el informe en qué casos convendría usar WorkManager en lugar de un Service iniciado manualmente para esta misma tarea de sincronización.

Entregable: informe con capturas del botón de sincronización, del resultado mostrado por la UI, del caso de detención manual, y una breve explicación de WorkManager como alternativa.

IV. CUESTIONARIO

1. ¿Lograste resolver completamente el Ejercicio/Problema Resuelto por el Docente (servicio en segundo plano que sincroniza datos y notifica el resultado mediante un BroadcastReceiver)? Adjunta una captura de pantalla del emulador mostrando el resultado como evidencia.
2. En tu propia implementación, ¿qué ocurriría si la tarea dentro de onStartCommand() se ejecutara directamente en el hilo principal en lugar de una coroutine en Dispatchers.IO? Explica el efecto sobre la interfaz de usuario.

V. REFERENCIAS Y BIBLIOGRÁFIA RECOMENDADAS:

[1] Android Developers, "Descripción general de servicios". [En línea]. Disponible: https://developer.android.com/develop/background-work/services

TÉCNICAS E INSTRUMENTOS DE EVALUACIÓN

TÉCNICAS:	**INSTRUMENTOS:**
Ejercicios propuestos	Rúbrica de evaluación

CRITERIOS DE EVALUACIÓN

- Cumplimiento de los objetivos y ejercicios propuestos en la guía.
- Correcta aplicación de los conceptos y herramientas del tema desarrollado.
- Funcionamiento correcto de lo implementado (compilación/ejecución sin errores).
- Informe (formato estudiante) completo, con evidencias y cuestionario resuelto.
- Entrega dentro del plazo establecido.
- Se permite el uso de Inteligencia Artificial (IA) como fuente de consulta o apoyo para resolver los ejercicios; sin embargo, el desarrollo completo del laboratorio no puede basarse únicamente en respuestas generadas por IA. El estudiante debe evidenciar comprensión propia mediante la justificación razonada de sus respuestas, especialmente en el cuestionario.